



山东大学

SDU OS Slide

Manual for the Shandong University OS slide template

Guide, examples, and API reference

PUBLIC SURFACE

setup()

pause

meanwhile

jump()

DESIGN TOKENS

theme.colors.primary

theme.page.sidebar-width

Guide

Guide	2
Quick Start	2
Slide Model	2
Setup Options	2
Theme Tokens	3
Overlays	3
Build Commands	4

Reference

Reference	5
slide	5
theme	5
overlays	6
foundation-theme	7
cover	8
sidebar	8
draw	10

Guide

SDU OS Slide recreates the visual language of the Shandong University operating-system course slides while keeping the authoring model deliberately small. A deck imports one setup function, writes ordinary Typst headings, and uses three overlay markers when a slide should build step by step.

What the template owns

`setup()` installs the cover, page frame, right sidebar, logo, heading behavior, raw-code styling, and overlay renderer. Your deck remains normal Typst content inside that frame.

Quick Start

```
#import "slide.typ": setup, pause, meanwhile, jump, theme
```

```
#show: setup.with(
  title: "SDU OS Slide",
  subtitle: "Slide for SDU OS",
  author: "arshtyi",
  term: "2026 Spring",
  date: datetime.today(),
  handout: false,
  sidebar-ring-style: 1,
)
```

= Chapter

== Slide Title

Always visible.

```
#pause
```

Visible on the next overlay.

Import path

Use a relative import while working from this repository.

Slide Model

The template uses Typst headings as slide boundaries:

- `= Section` starts a new slide and appears as a major sidebar entry.
- `== Slide` starts a new slide, appears in the sidebar, and is highlighted while active.
- `=== Topic` starts a new slide but is rendered as a centered in-slide title instead of a sidebar entry.
- Deeper headings inherit the slide-break rule, but they are best reserved for rare focus pages.

The right sidebar is built from outlined level 1 and level 2 headings. Each row is linked to the original heading location, so exported PDFs keep navigable section entries.

Setup Options

Name	Type / Default	Effect
<code>title</code>	<code>str</code> / "SDU OS Slide"	Main cover title.
<code>subtitle</code>	<code>str</code> / "Slide for SDU OS"	Cover subtitle and sidebar title.

Name	Type / Default	Effect
<code>author</code>	<code>str</code> / <code>"arshtyi"</code>	Cover badge and sidebar footer.
<code>term</code>	<code>str</code> / <code>"2026 Spring"</code>	Cover badge.
<code>date</code>	<code>datetime</code> / <code>datetime.today()</code>	Sidebar footer date, shown as <code>YYYY.MM</code> .
<code>handout</code>	<code>bool</code> / <code>false</code>	When true, each slide keeps only its final overlay state.
<code>sidebar-ring-style</code>	<code>int</code> / <code>1</code>	Controls the layering between the right sidebar and the lower-right decorative ring.

Sidebar ring styles

`sidebar-ring-style: 1` keeps the decorative ring above the sidebar table. `sidebar-ring-style: 2` recreates the original slide layering more closely: normal row fills sit below the ring, the active row sits above it, and gutters mask the ring.

Theme Tokens

Theme constants are exported as `theme` so user documents can read canonical values without copying magic numbers.

Token	Use
<code>theme.colors.primary</code>	SDU red for frame, cover badges, active sidebar rows, and headings.
<code>theme.colors.primary-soft</code>	Muted red for sidebar title rows and softer table fills.
<code>theme.colors.primary-muted</code>	Quiet sidebar row fill.
<code>theme.page.sidebar-width</code>	Reserved width for the right sidebar.
<code>theme.fonts.content</code>	Default body font pair used inside slides.

Overlays

`pause`, `meanwhile`, and `jump` are metadata markers processed by the overlay renderer. They do not draw anything by themselves.

Marker	Meaning
<code>#pause</code>	Advance by one overlay step. Later content is hidden until the next step.
<code>#meanwhile</code>	Reset following content back to overlay step 1. Useful for two parallel reveal sequences.
<code>#jump(n)</code>	Low-level overlay control. With <code>relative: true</code> , moves relative to the current step; otherwise jumps to an absolute step.

Always visible.

`#pause`

Reveal timeline

Visible on overlay 2.

```
#meanwhile
```

Also visible from overlay 1.

Step	Visible content
1	Always visible; meanwhile branch begins.
2	Everything from step 1 plus content after <code>pause</code> .

Handout build

Set `handout: true` to output only the final overlay state of each slide. This is useful for reading PDFs or course handouts.

Build Commands

The template is plain Typst. These are useful smoke tests before publishing changes:

```
typst compile --root . template/template.typ template/typst-sdu-os-slide.pdf
typst compile --root . docs/manual.typ docs/manual.pdf
```

Reference

slide

Public entry point. Most users import from this file only.

theme

Main setup function and page-level theme wiring.

```
setup()
```

setup()

Applies the SDU OS slide theme to the document.

The setup function renders a cover page, configures the slide canvas, installs the heading styles, and enables overlay rendering through `pause`, `meanwhile`, and `jump`.

Headings always start a new page. Level 3 and deeper headings are rendered as centered in-slide titles.

```
#import "slide.typ": setup, pause
```

```
#show: setup.with(
  title: "Operating Systems",
  subtitle: "Scheduling",
  author: "arshtyi",
  term: "2026 Spring",
)
```

```
= CPU Scheduling
```

```
== Round Robin
```

```
The ready queue is served by time slices.
```

```
#pause
```

```
A smaller quantum improves response time but raises switching overhead.
```

- `author (str)`: Author name shown on the cover and sidebar.
- `term (str)`: Course term shown on the cover.
- `title (str)`: Presentation title shown on the cover.
- `subtitle (str)`: Subtitle shown on the cover and sidebar.
- `date (datetime)`: Date shown in the sidebar.
- `handout (bool)`: Whether to render only the final overlay state of each slide.
- `sidebar-ring-style (int)`: `1` draws the decorative ring above the sidebar; `2` interleaves the ring with sidebar fills, active row, and gutters.
- `body (content)`: Presentation body.

Parameters

```
setup(
  author,
  term,
  title,
  subtitle,
  date,
  handout,
  sidebar-ring-style,
  body
) -> content
```

overlays

Overlay markers and the renderer used by `setup`.

```
jump()
render-overlays()
```

Variables

```
pause
meanwhile
```

`jump()`

Creates an overlay jump marker.

`jump` is the common primitive behind `pause` and `meanwhile`. With `relative: true`, `n` moves the current overlay step by a relative amount. With `relative: false`, `n` sets the current overlay step absolutely.

Always visible.

```
#jump(2)
```

Hidden on overlay 1, visible from overlay 2.

- `n` (int): Relative or absolute overlay step.
- `relative` (bool): Whether `n` is a relative offset. Defaults to `false`.

Parameters

```
jump(n, relative) -> content
```

`render-overlays()`

Renders a document body with slide overlays.

The renderer splits the body at headings and at explicit `pagebreak()` calls. In presentation mode, each resulting slide is duplicated once for each overlay step introduced by `pause`, `meanwhile`, or `jump`. In handout mode, only the final overlay state of each slide is rendered.

This is normally called by `setup()`; user documents usually call the overlay markers instead.

- `handout` (bool): Whether to render only final overlay states. Defaults to `false`.
- `body` (content): Document body to render.

Parameters

```
render-overlays(body, handout) -> content
```

pause

```
content
```

Advances to the next overlay step.

Content after `pause` is hidden on the current overlay and appears on the next overlay. It is equivalent to `jump(1, relative: true)`.

```
First point.
```

```
#pause
```

```
Second point.
```

meanwhile

```
content
```

Resets subsequent content to the first overlay step.

Content after `meanwhile` appears together with the beginning of the slide, while earlier paused content keeps its own reveal timing. It is equivalent to `jump(1)`.

```
Left story.
```

```
#pause
```

```
Left story, step 2.
```

```
#meanwhile
```

```
Right story starts on overlay 1.
```

foundation-theme

Shared fonts, colors, and page geometry.

Variables

```
theme
```

theme

```
dictionary
```

Shared visual constants for the SDU OS slide theme.

The dictionary contains font families, colors, and page dimensions used by the cover, sidebar, frame, and body text.

```
#import "slide.typ": theme
```

```
#text(fill: theme.colors.primary)[Important]
```

```
#rect(width: theme.page.sidebar-width, height: 1cm)
```

cover

Cover-page component.

```
render-cover()
```

render-cover()

Renders the opening cover page.

The cover uses the SDU and SDU CS logos, decorative circles, and the theme's cover font and color tokens. It also resets the page counter so the first content slide starts from page 1.

- title (str): Main title shown in the center of the cover.
- subtitle (str): Subtitle shown below the title.
- author (str): Author name shown in the lower badge.
- term (str): Course term shown in the lower badge.

Parameters

```
render-cover(  
  title,  
  subtitle,  
  author,  
  term  
) -> content
```

sidebar

Right-side outline, active-row detection, and sidebar/ring layering helpers.

```
format-outline-heading()  
find-active-subheading()  
build-outline-cells()  
render-sidebar()  
render-sidebar-gap-mask()
```

format-outline-heading()

Formats a level 1 or level 2 heading for the sidebar outline.

The generated heading text is linked to the original heading location. Level 1 headings use bold body text; level 2 headings keep regular body text after the numbering.

- heading-node (content): Heading node returned by `query(heading)`.

Parameters

```
format-outline-heading(heading-node) -> content
```

find-active-subheading()

Finds the active level 2 heading for a page.

A subheading is active from the page where it appears until the next outline heading starts. This keeps overlay pages inside the same highlighted sidebar row.

- outline-headings (array): Queried level 1 and level 2 headings.
- current-page (int): Current page number.

Parameters

```
find-active-subheading(outline-headings, current-page) -> none content
```

build-outline-cells()

Builds table cells for the sidebar outline.

The returned dictionary contains the cells and the table row that should be filled with the active color.

- `outline-headings` (array): Queried level 1 and level 2 headings.
- `active-subheading` (none, content): Active level 2 heading, if any.
- `show-text` (bool): Whether to render the row labels.

Parameters

```
build-outline-cells(
  outline-headings,
  active-subheading,
  show-text
) -> dictionary
```

render-sidebar()

Renders a sidebar table layer.

The `fill-mode` controls which row backgrounds are drawn: `"full"` draws the normal sidebar fills, `"base"` skips the active row, and `"active"` only draws the active row fill. This lets the theme interleave the decorative ring between normal row fills and the active row.

`setup()` calls this internally. It is documented because the sidebar is the most template-specific component and is useful when customizing the theme.

- `title` (str): Sidebar title, usually the presentation subtitle.
- `author` (str): Author name shown in the footer.
- `date` (datetime): Date shown in the footer.
- `fill-mode` (str): Either `"full"`, `"base"`, or `"active"`.
- `show-text` (bool): Whether to render title, outline labels, and footer.

Parameters

```
render-sidebar(
  title,
  author,
  date,
  fill-mode,
  show-text
) -> content
```

render-sidebar-gap-mask()

Masks sidebar gutters above the decorative ring.

The mask follows the same row-gutter sequence as the sidebar table and adds a thin right and bottom edge so the ring remains clipped by the table gaps.

- `width` (length): Width of the sidebar block.
- `height` (length): Height of the sidebar block.

- `edge-gutter` (length): Thickness of the right and bottom masks.

Parameters

```
render-sidebar-gap-mask(  
  width,  
  height,  
  edge-gutter  
) -> content
```

draw

Low-level drawing helpers.

```
place-circle()
```

place-circle()

Places a CeTZ circle by its center point.

Typst's `place` works from a top-left anchor in this template, while the visual design is easier to describe with circle centers. This helper converts center coordinates and radius to the correct placed CeTZ canvas.

- `center-x` (length): Horizontal center position.
- `center-y` (length): Vertical center position.
- `radius` (length): Circle radius.
- `style` (arguments): Additional CeTZ circle styling such as `fill` or `stroke`.

Parameters

```
place-circle(  
  center-x,  
  center-y,  
  radius,  
  ..style  
) -> content
```